

Report sul progetto di Machine Learning per la predizione dei campioni futuri delle serie storiche di EXXON e SHELL (MACCA-CIAMPI)

Fonte serie storiche: [Nasdaq.com](https://www.nasdaq.com)

Abbiamo scelto ExxonMobil (XOM) e Royal Dutch Shell (SHELL). Entrambe le aziende sono leader mondiali nel settore energetico integrato, con una forte attenzione sia alla filiera tradizionale degli idrocarburi sia alla diversificazione verso soluzioni più sostenibili. Questo mercato è notoriamente di tipo oligopolistico e le grandi società che operano in esso hanno fra di loro una fortissima correlazione causata dalle fluttuazioni del prezzo del greggio, decisioni geopolitiche (un esempio è l'Opec) e dal ciclo economico globale; essendo entrambe attive in tutto il mondo, le due società risentono allo stesso modo alle crisi o crescite economiche.

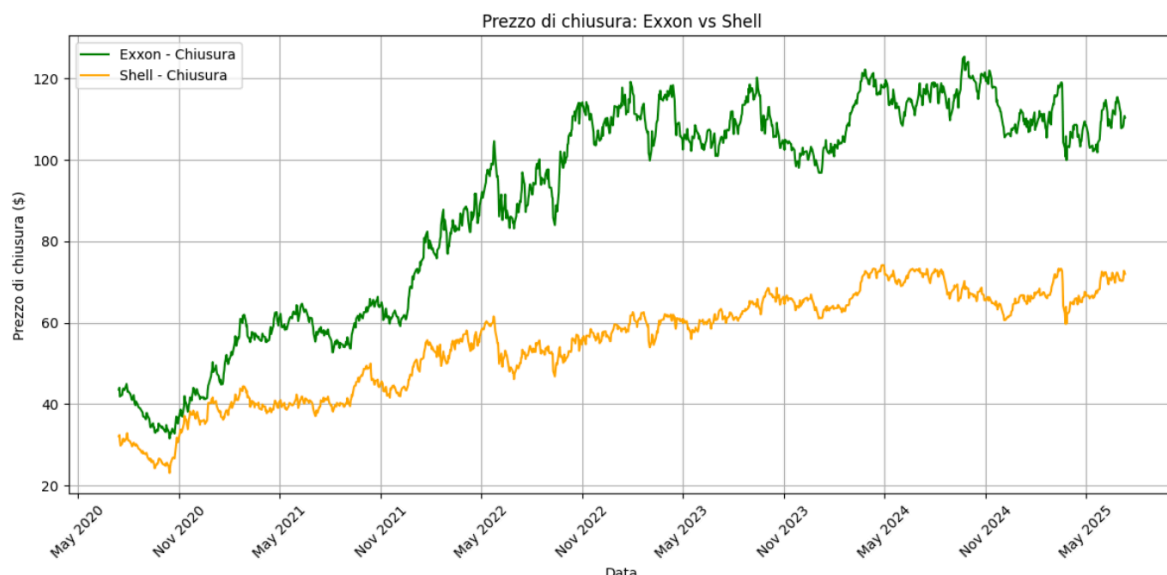
L'obiettivo di questa analisi è la predizione delle serie storiche dei due titoli utilizzando metodi di machine learning.

Analisi descrittiva dei dati

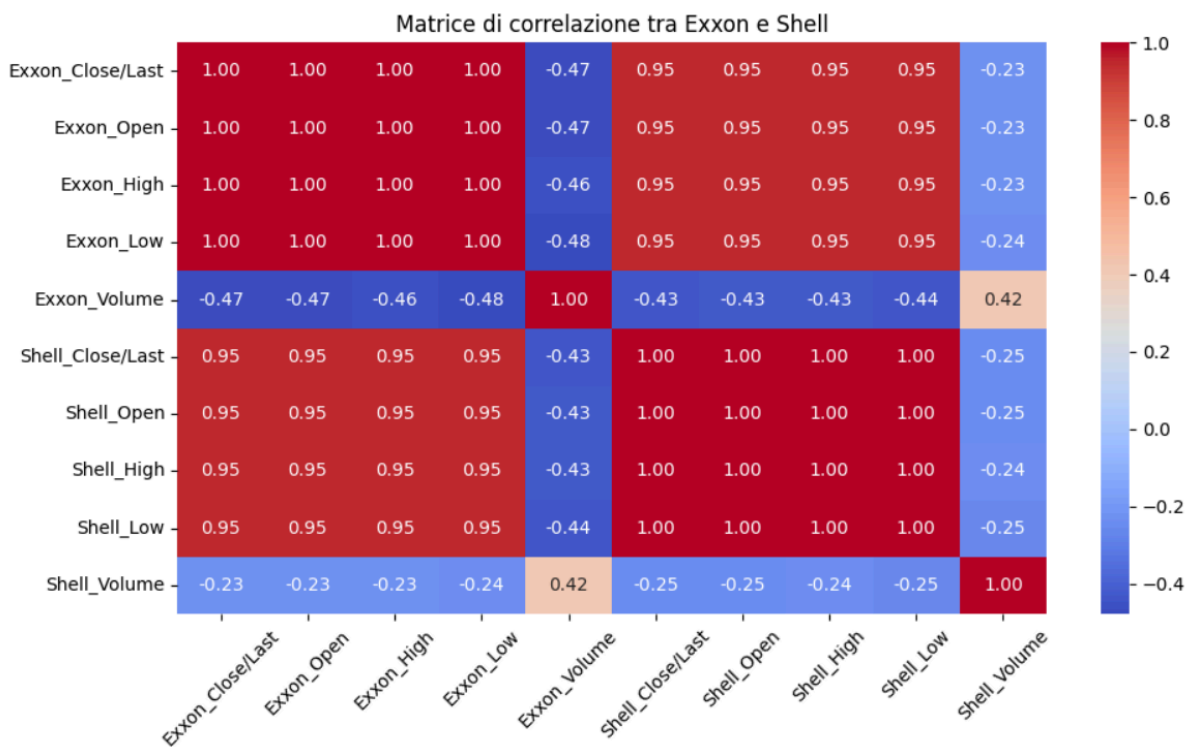
I dataset utilizzati comprendono le serie storiche in borsa di Exxon e Shell che contengono prezzi di apertura e chiusura, massimi, minimi e volumi di scambio.

La nostra analisi ha evidenziato:

- Entrambe le serie mostrano un andamento estremamente simile fra loro, suggerendo una correlazione molto positiva.

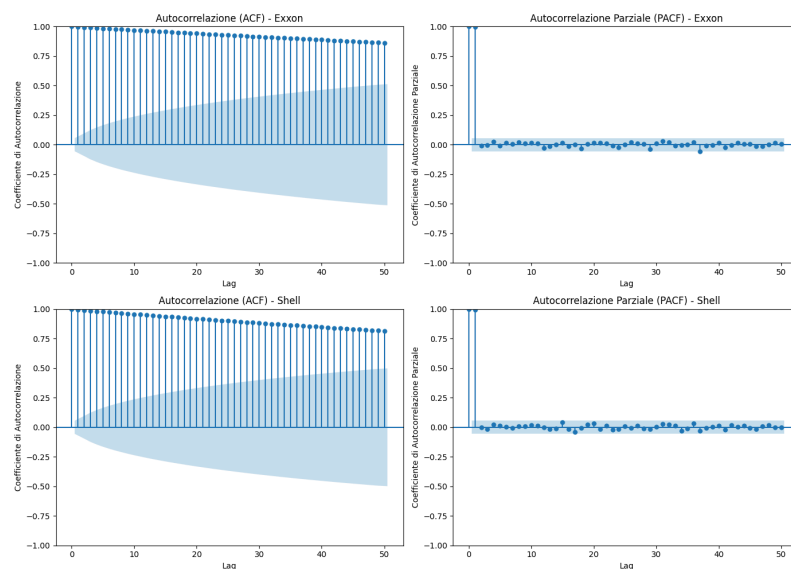


La Matrice di correlazione evidenzia come i due si muovono all'unisono con correlazioni vicine a 1 guardando i prezzi di apertura e di chiusura. Mentre invece la relazione della liquidità scambiata fra i due è più debole.



Grafici di autocorrelazione (ACF) e autocorrelazione parziale (PACF)

I prezzi di Exxon e Shell hanno una forte auto-correlazione in particolare sono fortemente dipendenti dal giorno precedente.

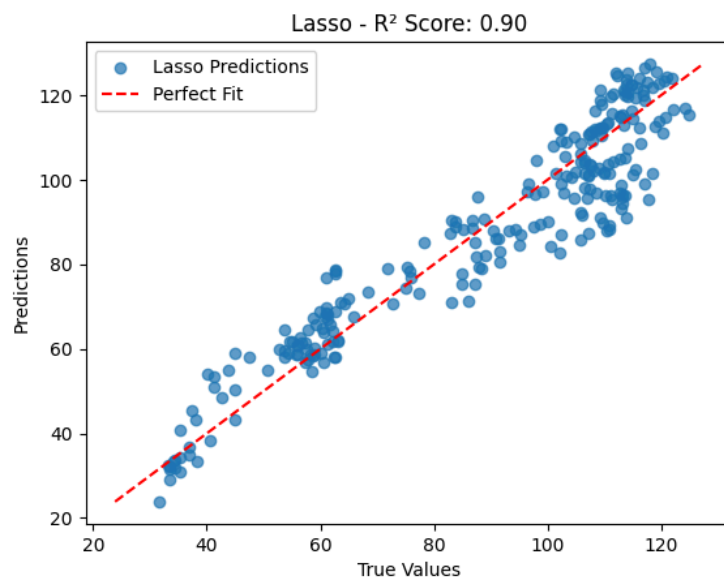


MODELLI DI MACHINE LEARNING

Abbiamo applicato tre diversi approcci per la predizione dei prezzi di chiusura: Regressione Lasso, XGBoost e LSTM.

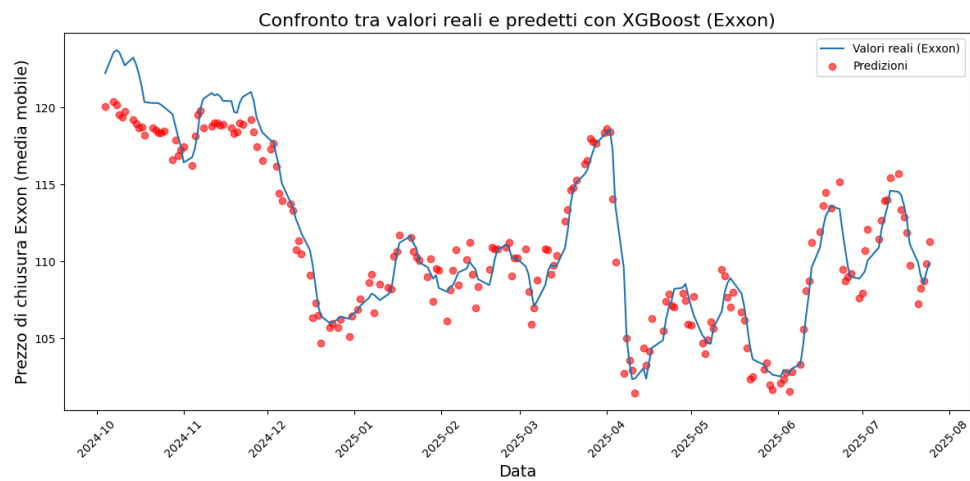
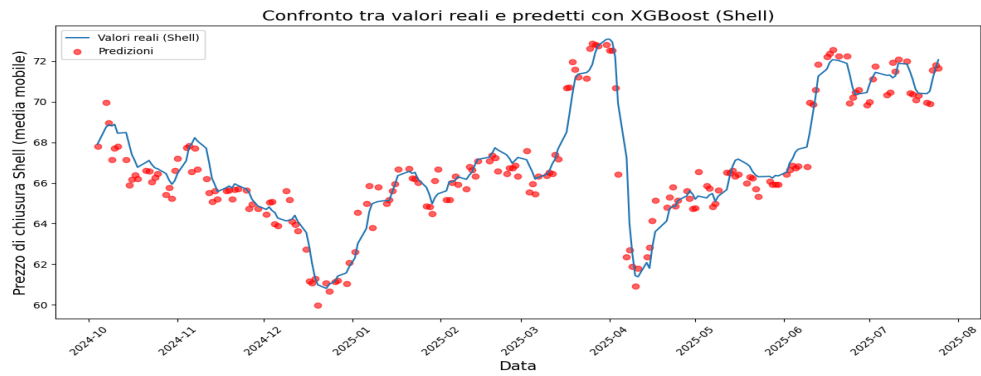
REGRESSIONE LASSO (EXXON)

Dal grafico si evince che i dati predetti dal modello sono molto vicini a quelli reali, confermando una buona capacità di previsione. Si nota comunque un po' di dispersione nei valori più alti, alcune predizioni tendono a sovrastimare o sottostimare rispetto alla *Perfect Fit*. In generale il modello predittivo è solido e affidabile.



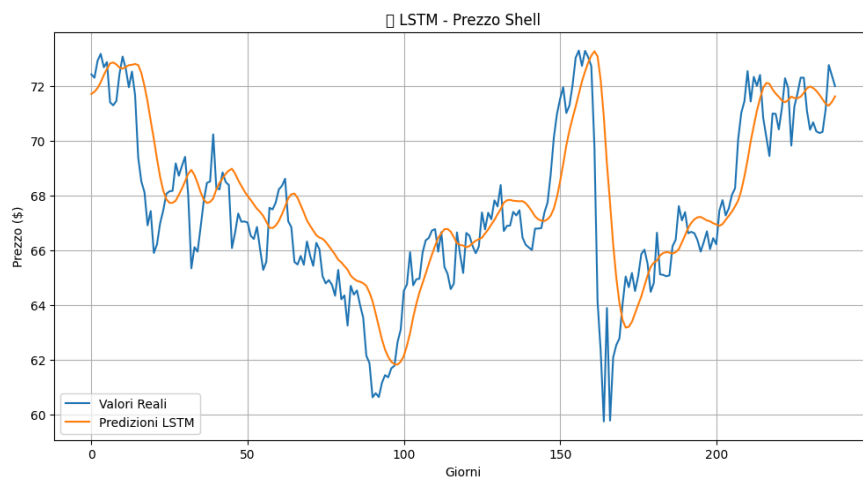
XGBOOST

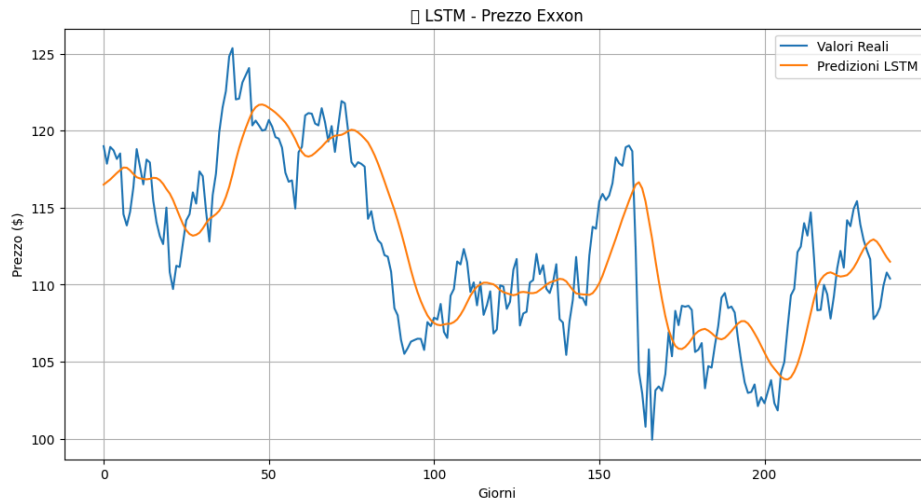
Questo metodo prevede abbastanza bene il trend dei prezzi reali, la correlazione è molto buona e cattura i movimenti principali, specialmente nei periodi stabili come Gennaio-Marzo 2025. Notiamo delle differenze fra la previsione di Exxon e di Shell soprattutto nella maggiore precisione nei picchi e nei range di prezzo di Shell; possiamo concludere che il metodo XGBoost ha performato leggermente meglio sui dati di Shell. Questo potrebbe dipendere dalle caratteristiche intrinseche dei dati di shell che potrebbero rendere i suoi movimenti più prevedibili.



LONG SHORT-TERM MEMORY

In questo modello di rete neurale abbiamo basato la predizione su un lasso di tempo di 250 giorni.





```
LSTM Metrics (Exxon):  
MSE: 11.072378359709663  
MAE: 2.5909903145634483  
R²: 0.6550701067024188
```

```
LSTM Metrics (Shell):  
MSE: 2.9786471568277055  
MAE: 1.1793852010431647  
R²: 0.681576446321762
```

I grafici mostrano che il prezzo delle azioni di Exxon è più volatile rispetto a quello di Shell. Le fluttuazioni sono molto simili in quanto sono due aziende operanti nello stesso settore e risentono degli stessi fattori macroeconomici e di mercato.

Il modello ha performato leggermente meglio con i dati provenienti da Shell, con R^2 di 68.2% rispetto al 65.5% di Exxon, anche l'errore assoluto medio (MAE) di Shell è più basso di Exxon, con 1.18 (la previsione si discosta dal prezzo reale di circa \$1.18) e 2.59.

Il modello LSTM ha mostrato una performance simile per le due aziende, con un leggero vantaggio per Shell, i valori di R^2 indicano che il modello cattura una porzione significativa della variabilità dei dati.

CONCLUSIONE

Dai risultati emerge che XGBoost offre la migliore previsione dei prezzi futuri in quanto riesce a mantenere performance alte per entrambe le aziende catturando sia trend stabili che picchi cosa in cui la regressione lasso pecca. LSTM ha un R^2 di molto inferiore evidenziando difficoltà nell'apprendere le dinamiche di prezzo.

Di seguito il codice utilizzato per l'analisi svolta.

```

import tensorflow as tf
import pandas as pd #gestione dati tabellari
import numpy as np #operazioni numeriche
import matplotlib.pyplot as plt #grafiche base
import matplotlib.dates as mdates #gestione date nei grafici
import seaborn as sns #grafici avanzati e heatmap
import random
import math
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf #autocorrelazione
import xgboost as xgb

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler #questo e quello sopra per preparazione dati per i modelli

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor

from sklearn import linear_model #modelli e regressioni classici anche sopra
from sklearn.metrics import r2_score, mean_squared_error #commentare il modello
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.optimizers import AdamW #deep learning

#statistiche descrittive:
# Carica i file CSV nelle rispettive variabili
HistoricalDataExxon = pd.read_csv("HistoricalDataExxon.csv")
HistoricalDataShell = pd.read_csv("HistoricalDataShell.csv")
print(HistoricalDataExxon.columns)
print(HistoricalDataShell.columns)
#verifico che le colonne siano numeriche:
print(HistoricalDataExxon.dtypes)
print(HistoricalDataShell.dtypes)

```

```

print(HistoricalDataShell.dtypes)
#convertito in float
# Rimuovo il simbolo "$" e converte le stringhe in float
for col in ['Close/Last', 'Open', 'High', 'Low']:
    HistoricalDataExxon[col] = HistoricalDataExxon[col].replace(['$'], '', regex=True).astype(float)
    HistoricalDataShell[col] = HistoricalDataShell[col].replace(['$'], '', regex=True).astype(float)

print(HistoricalDataExxon.describe())
print(HistoricalDataShell.describe())

# Conversione del campo 'Date' in formato datetime
HistoricalDataExxon['Date'] = pd.to_datetime(HistoricalDataExxon['Date'])
HistoricalDataShell['Date'] = pd.to_datetime(HistoricalDataShell['Date'])

# Creazione del grafico
plt.figure(figsize=(12, 6))

#Linee dei prezzi di chiusura
plt.plot(HistoricalDataExxon['Date'], HistoricalDataExxon['Close/Last'], label="Exxon - Chiusura", color='red')
plt.plot(HistoricalDataShell['Date'], HistoricalDataShell['Close/Last'], label="Shell - Chiusura", color='orange')

#Titoli e assi
plt.title("Prezzo di chiusura: Exxon vs Shell")
plt.xlabel("Data")
plt.ylabel("Prezzo di chiusura ($)")
plt.legend()

# Etichette sull'asse X
plt.gca().xaxis.set_major_locator(mdates.MonthLocator(bymonthdays=15, interval=6))
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))

#Matrice di correlazione
# Selezione delle colonne numeriche e rinomina
exxon = HistoricalDataExxon[['Date', 'Close/Last', 'Open', 'High', 'Low', 'Volume']].copy()
exxon.columns = ['Date'] + [f'Exxon_{col}' for col in Exxon.columns[1:]]

```

```

# PACF per Exxon
plot_pacf(exxon_close, lags=50, ax=axes[0, 1])
axes[0, 1].set_title("Autocorrelazione Parziale (PACF) - Exxon")
axes[0, 1].set_xlabel("Lag")
axes[0, 1].set_ylabel("Coefficiente di Autocorrelazione Parziale")

# ACF per Shell
plot_acf(shell_close, lags=50, ax=axes[1, 0])
axes[1, 0].set_title("Autocorrelazione (ACF) - Shell")
axes[1, 0].set_xlabel("Lag")
axes[1, 0].set_ylabel("Coefficiente di Autocorrelazione")

# PACF per Shell
plot_pacf(shell_close, lags=50, ax=axes[1, 1])
axes[1, 1].set_title("Autocorrelazione Parziale (PACF) - Shell")
axes[1, 1].set_xlabel("Lag")
axes[1, 1].set_ylabel("Coefficiente di Autocorrelazione Parziale")

# Mostra i grafici
plt.tight_layout()
plt.show()

print(merged_data.info())
print(merged_data.head())

#REGRESSIONE LASSO
# Crea un dizionario per salvare i risultati
lst = {}

# Copia il dataset originale
df = merged_data.copy()

# Rimuove eventuali colonne non necessarie (modifica se vuoi escludere altre)
colonne_da_eliminare = ['Exxon_Open', 'Exxon_High', 'Exxon_Low', 'Date'] # example: also remove 'Date'
df = df.drop(colonne_da_eliminare, axis=1)

# Suddivide il dataset in training (80%) e test (20%)
df_train, df_test = train_test_split(df, train_size=0.8, random_state=42)

# Estrae la variabile target e le feature
y_train = df_train.pop('Exxon_Close/Last')
x_train = df_train
y_test = df_test.pop('Exxon_Close/Last')
x_test = df_test

```

```

shell = HistoricalDataShell[['Date', 'Close/Last', 'Open', 'High', 'Low', 'Volume']].copy()
shell.columns = ['Date'] + [f'Shell_{col}' for col in shell.columns[1:]]

# Concatenazione dei dataset
merged_data = pd.concat([exxon, shell.drop('Date', axis=1)], axis=1)

# Calcolo e visualizzazione della matrice di correlazione
plt.figure(figsize=(10, 6))
sns.heatmap(merged_data.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('🔗 Matrice di correlazione tra Exxon e Shell')
plt.xticks(rotation=45)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

#ANALISI TEMPORALE
#ACF: quanto una variabile è simile a sé stessa nel tempo
#la metto perché è utile per LSTM
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# 1. Estrai le serie temporali dei prezzi di chiusura
exxon_close = HistoricalDataExxon["Close/Last"]
shell_close = HistoricalDataShell["Close/Last"]

# 2. Crea i grafici
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# ACF per Exxon
plot_acf(exxon_close, lags=50, ax=axes[0, 0])
axes[0, 0].set_title("Autocorrelazione (ACF) - Exxon")
axes[0, 0].set_xlabel("Lag")
axes[0, 0].set_ylabel("Coefficiente di Autocorrelazione")

```



```

# Modello di regressione lineare
linear_model = LinearRegression()
linear_model.fit(x_train, y_train)
y_pred_linear = linear_model.predict(x_test)

# Modello Lasso
lasso_model = Lasso(alpha=0.5)
lasso_model.fit(x_train, y_train)
y_pred_lasso = lasso_model.predict(x_test)

# Calcola R^2 e salva nel dizionario
lst['Lasso'] = r2_score(y_test, y_pred_lasso)
print("R² Score Lasso:", lst['Lasso'])

# Grafico a dispersione
fig, ax = plt.subplots()
ax.scatter(y_test, y_pred_lasso, label='Lasso Predictions', alpha=0.7)

# Linea perfetta y = x
min_val = min(np.min(y_test), np.min(y_pred_lasso))
max_val = max(np.max(y_test), np.max(y_pred_lasso))
perfect_fit_line = np.linspace(min_val, max_val, 100)
ax.plot(perfect_fit_line, perfect_fit_line, color='red', linestyle='--', label='Perfect Fit')

ax.set_title('Lasso - R² Score: {:.2f}'.format(lst['Lasso']))
plt.xlabel('True Values')
plt.ylabel('Predictions')
plt.legend()
plt.show()

#XGBoost non mi viene, abbiamo un problema con la colonna Date
# Import necessari
from xgboost import XGBRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt
import pandas as pd

#colonna Date nei dataset
exxon['Date'] = pd.to_datetime(HistoricalDataExxon['Date'], errors='coerce')
shell['Date'] = pd.to_datetime(HistoricalDataShell['Date'], errors='coerce')

#Aggrega i dati per ogni data
exxon_grouped = exxon.groupby('Date').agg({
    'Exxon_Close/Last': 'mean',
    'Exxon_Open': 'mean',
    'Exxon_High': 'mean',
    'Exxon_Low': 'mean',
    'Exxon_Volume': 'sum'
}).reset_index()

```

```

shell_grouped = shell.groupby('Date').agg({
    'Shell_Close/Last': 'mean',
    'Shell_Open': 'mean',
    'Shell_High': 'mean',
    'Shell_Low': 'mean',
    'Shell_Volume': 'sum'
}).reset_index()

# Rinomina le colonne per identificare l'origine
exxon_grouped.columns = ['Date'] + [col for col in exxon_grouped.columns[1:]]
shell_grouped.columns = ['Date'] + [col for col in shell_grouped.columns[1:]]

# Merge dei due dataset sulle date
merged_data = pd.merge(exxon_grouped, shell_grouped, on='Date', how='inner')

# Ordina cronologicamente
merged_data = merged_data.sort_values(by='Date')

# Calcolo della media mobile a 4 giorni
merged_data['Exxon_SMA'] = merged_data['Exxon_Close/Last'].rolling(window=4).mean()
merged_data['Shell_SMA'] = merged_data['Shell_Close/Last'].rolling(window=4).mean()

# Rimuove righe con values NaN
merged_data = merged_data.dropna()

# Prepara le feature per il modello
features = merged_data.drop(columns=['Date', 'Exxon_Close/Last', 'Shell_Close/Last', 'Exxon_SMA', 'Shell_SMA'])

# Funzione per allenare e visualizzare XGBoost
def train_xgb(X, y, label):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.16, shuffle=False)
    model = XGBRegressor(
        n_estimators=300,
        learning_rate=0.03,
        max_depth=6,
        subsample=0.8,
        colsample_bytree=0.8,
        objective="reg:squarederror",
        random_state=42
    )
    model.fit(X_train, y_train)
    y_pred_train = model.predict(X_train)
    y_pred_test = model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred_test)
    print(f"Mean Squared Error ({label}): {mse:.4f}")

# Grafico del confronto
plt.figure(figsize=(12, 6))
plt.plot(merged_data['Date'].iloc[len(y_train):len(y_train)+len(y_test)], y_test.values, label=f"Valori reali ({label})")
plt.scatter(merged_data['Date'].iloc[len(y_train):len(y_train)+len(y_test)], y_pred_test, label="Predizioni", color="red", alpha=0.6)
plt.xlabel("Data", fontsize=14)
plt.ylabel(f"Prezzo di chiusura {label} (media mobile)", fontsize=14)
plt.title(f"Confronto tra valori reali e predetti con XGBoost ({label})", fontsize=16)
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
plt.show()

```

```

        return {'y_train_actual': y_train, 'train_predictions': y_pred_train, 'y_test_actual': y_test, 'test_predictions': y_pred_test}

def calculate_metrics(results, company_name):
    """
    Calcola e stampa le metriche di performance
    """
    from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

    print(f"\n Metriche per {company_name}:")
    print("-" * 40)

    # Metriche sul training set
    train_mse = mean_squared_error(results['y_train_actual'], results['train_predictions'])
    train_mae = mean_absolute_error(results['y_train_actual'], results['train_predictions'])
    train_r2 = r2_score(results['y_train_actual'], results['train_predictions'])

    print(f"Training Set:")
    print(f"  MSE: {train_mse:.4f}")
    print(f"  MAE: {train_mae:.4f}")
    print(f"  R²: {train_r2:.4f}")

    # Metriche sul test set
    test_mse = mean_squared_error(results['y_test_actual'], results['test_predictions'])
    test_mae = mean_absolute_error(results['y_test_actual'], results['test_predictions'])
    test_r2 = r2_score(results['y_test_actual'], results['test_predictions'])

    print(f"Test Set:")
    print(f"  MSE: {test_mse:.4f}")
    print(f"  MAE: {test_mae:.4f}")
    print(f"  R²: {test_r2:.4f}")

# Esegui training su Exxon
X_exxon = features.copy()
y_exxon = merged_data['Exxon_SMA']
exxon_results = train_xgb(X_exxon, y_exxon, label="Exxon")

# Esegui training su Shell
X_shell = features.copy()
y_shell = merged_data['Shell_SMA']
shell_results = train_xgb(X_shell, y_shell, label="Shell")

# Stampa le metriche
calculate_metrics(exxon_results, "Exxon")
calculate_metrics(shell_results, "Shell")

# Usa i dati già puliti e ordinati
data = merged_data[['Exxon_Close/Last']].copy()

# Normalizzazione
scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(data)

```

```

# Funzione per creare sequenze
def create_sequences(data, seq_length):
    x = []
    y = []
    for i in range(seq_length, len(data)):
        x.append(data[i-seq_length:i, 0])
        y.append(data[i, 0])
    return np.array(x), np.array(y)

# Parametri
sequence_length = 60 # giorni di storico usati per predire il successivo

# Crea sequenze
X, y = create_sequences(data_scaled, sequence_length)

# Ridimensiona per LSTM |
X = X.reshape((X.shape[0], X.shape[1], 1))

# Divisione train/test
split = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

model = Sequential()
model.add(LSTM(units=64, return_sequences=True, input_shape=(X_train.shape[1], 1)))
model.add(Dropout(0.2))
model.add(LSTM(units=64))
model.add(Dropout(0.2))
model.add(Dense(1))

model.compile(optimizer='adam', loss='mean_squared_error')
model.summary()

early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=1
)

# Predizione
y_pred_scaled = model.predict(X_test)
y_pred = scaler.inverse_transform(y_pred_scaled)
y_test_actual = scaler.inverse_transform(y_test.reshape(-1, 1))

```

```

# Grafico confronto
plt.figure(figsize=(12,6))
plt.plot(y_test_actual, label='Valori Reali')
plt.plot(y_pred, label='Predizioni LSTM')
plt.title('LSTM - Prezzo Exxon')
plt.xlabel('Giorni')
plt.ylabel('Prezzo ($)')
plt.legend()
plt.grid(True)
plt.show()

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

print("LSTM Metrics (Exxon):")
print("MSE:", mean_squared_error(y_test_actual, y_pred))
print("MAE:", mean_absolute_error(y_test_actual, y_pred))
print("R²:", r2_score(y_test_actual, y_pred))

# LSTM PER SHELL
# Usa i dati già puliti e ordinati
data_shell = merged_data[['Shell_Close/Last']].copy()

# Normalizzazione
scaler_shell = MinMaxScaler()
data_scaled_shell = scaler_shell.fit_transform(data_shell)

# Crea sequenze for Shell
X_shell_lstm, y_shell_lstm = create_sequences(data_scaled_shell, sequence_length)

# Ridimensiona per LSTM [samples, timesteps, features]
X_shell_lstm = X_shell_lstm.reshape((X_shell_lstm.shape[0], X_shell_lstm.shape[1], 1))

# Divisione train/test
split_shell = int(0.8 * len(X_shell_lstm))
X_train_shell, X_test_shell = X_shell_lstm[:split_shell], X_shell_lstm[split_shell:]
y_train_shell, y_test_shell = y_shell_lstm[:split_shell], y_shell_lstm[split_shell:]

# Modello LSTM per Shell
model_shell = Sequential()
model_shell.add(LSTM(units=64, return_sequences=True, input_shape=(X_train_shell.shape[1], 1)))
model_shell.add(Dropout(0.2))
model_shell.add(LSTM(units=64))
model_shell.add(Dropout(0.2))
model_shell.add(Dense(1))

model_shell.compile(optimizer='adam', loss='mean_squared_error')

early_stop_shell = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

history_shell = model_shell.fit(
    X_train_shell, y_train_shell,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stop_shell],
    verbose=1
)

# Predizione Shell
y_pred_scaled_shell = model_shell.predict(X_test_shell)
y_pred_shell = scaler_shell.inverse_transform(y_pred_scaled_shell)
y_test_actual_shell = scaler_shell.inverse_transform(y_test_shell.reshape(-1, 1))

```

```
# Grafico confronto Shell
plt.figure(figsize=(12,6))
plt.plot(y_test_actual_shell, label='Valori Reali')
plt.plot(y_pred_shell, label='Predizioni LSTM')
plt.title('LSTM - Prezzo Shell')
plt.xlabel('Giorni')
plt.ylabel('Prezzo ($)')
plt.legend()
plt.grid(True)
plt.show()

# Metriche Shell
print("LSTM Metrics (Shell):")
print("MSE:", mean_squared_error(y_test_actual_shell, y_pred_shell))
print("MAE:", mean_absolute_error(y_test_actual_shell, y_pred_shell))
print("R²:", r2_score(y_test_actual_shell, y_pred_shell))
```